

Module 2: Frontend Responsive Design for Chatbots

Objective:

By the end of this session, you will:

- Understand how to build an adaptive chat UI for web and mobile.
- Learn how to implement live message streaming and typing effects.
- Master component architecture using React, Tailwind CSS, and Next.js.
- Design accessibility and user-centric UX patterns for chatbots.

1. Understanding Responsive Chat UI

A responsive chatbot interface automatically adjusts layout, font size, and alignment depending on device width and orientation.

Over 70% of chatbot interactions occur on mobile devices.

Key Responsive Elements:

- Chat Bubbles: Rounded, color-differentiated message containers.
- Input Bar: Fixed at bottom, multiline support, responsive.
- Scrollable History: Smooth auto-scroll on new message.
- Typing Indicator: Animated dots.
- Quick Replies: Suggestion buttons that adapt to screen width.

2. Mobile-First Layout Strategy

Design for small screens first, then enhance for larger screens.

Breakpoints (TailwindCSS style):

- <640px → Single column chat view.
- 641–1024px → Two columns: chat + context panel.
- >1024px → Three columns: chat | context | analytics.

3. Accessibility & UX Guidelines

Accessibility ensures usability for everyone.

Guidelines:

- Semantic HTML: , , .
- ARIA Labels: “Send message”, “Open menu”.
- Keyboard Navigation: Tab focus through controls.
- Contrast: Maintain $\geq 4.5:1$ ratio.
- Respect prefers-reduced-motion for animations.

4. Real-Time Message Streaming

Modern chatbots stream responses token-by-token for realism.

Frontend (React):

```
useEffect(() => {  
  const evtSource = new EventSource("/chat/stream");  
  evtSource.onmessage = (event) => {  
    setMessages((msgs) => [...msgs, JSON.parse(event.data)]);  
  };  
  return () => evtSource.close();  
}, []);
```

Backend (FastAPI):

```
@app.get("/chat/stream")
```

async def stream_chat():

async def event_generator():

for token in generate_response():

```
yield f"data: {json.dumps({'content': token})}\n\n"  
return StreamingResponse(event_generator(), media_type="text/event-stream")
```

5. Component Architecture

Decompose UI into modular React components:

Components:

- ChatContainer.jsx
- MessageList.jsx
- MessageBubble.jsx
- InputBar.jsx
- TypingIndicator.jsx
- Sidebar.jsx

State Flow:

User input → sendMessage() → API → stream → update messageList[]

Use useState/useReducer or Zustand/Redux Toolkit for scalable state.

6. UX Enhancements

Micro-interactions:

- Fade-in messages, auto-scroll, animated send button.

Optional Add-ons:

- Sound cues, haptics, shimmer loading placeholders.

7. Edge Case Handling

- Network Loss → “Reconnecting...” toast.
- Empty Input → Disable send button.
- Long Messages → “Read more” toggle.
- Back Navigation → Persist in localStorage.
- Theme Toggle → Dark/Light mode.

8. Folder Structure Example

/chatbot-ui

■ /components

■ ■ ChatContainer.jsx

■ ■ MessageList.jsx

■ ■ MessageBubble.jsx

■ ■ InputBar.jsx

■ ■ TypingIndicator.jsx

■ /hooks

■ ■ useChatStream.js

■ /pages

■ ■ index.jsx

■ /styles

■ ■ globals.css

■ tailwind.config.js

Summary:

- Responsive Layouts: Mobile-first, adaptive columns.
- Streaming: Real-time via SSE/WebSocket.
- Accessibility: ARIA + semantic HTML.
- Modular Components: For scalability.
- UX Details: Auto-scroll, typing animation, quick replies.

Practice Project:

Goal: Build a responsive React chatbot UI connected to FastAPI.

Tasks:

1. Create layout using Tailwind.
2. Implement message streaming.
3. Add typing animation.
4. Store chat in localStorage.
5. Deploy on Vercel (frontend) + Render/AWS (backend).